UNITÀ 1 · CODIFICA DELLE INFORMAZIONI

# Numeri con la virgola

Perché per un computer 0,1 + 0,2 non fa esattamente 0,3? Viaggio tra virgola fissa, virgola mobile e lo standard IEEE 754.

Virgola fissa | Virgola mobile | IEEE 754

PER COMINCIARE

# Per un computer, 0,1 + 0,2 fa 0,3?

SONDAGGIO

- **A** Sì, sempre: i computer non sbagliano i conti
- **B** No, mai
- **C** Dipende da come il numero è memorizzato
- **D** Solo se il computer è vecchio

DOPO LA VIRGOLA

# In binario i pesi continuano a dimezzarsi

| 1 | 0 | 1 | , | 1 | 0 | 1 |
|---|---|---|---|---|---|---|
| 4 | 2 | 1 | | 0,5 | 0,25 | 0,125 |

101,101 in base 2

$$4 + 1 + 0{,}5 + 0{,}125 = 5{,}625$$

Dopo la virgola i pesi sono $2^{-1} = 0{,}5 \cdot 2^{-2} = 0{,}25 \cdot 2^{-3} = 0{,}125 \cdot 2^{-4} = 0{,}0625$ …

DA DECIMALE A BINARIO

# La parte dopo la virgola: moltiplica per 2

**0,375**

0,375 × 2 = **0**,75

0,75 × 2 = **1**,5

0,5 × 2 = **1**,0 → fine

Leggi le cifre dall'alto: **0,011**

**0,1**

0,1 → 0,2 → 0,4 → 0,8 → 1,6 → 1,2 → 0,4 → 0,8 → 1,6 …

Cifre: **0,0001100110011…**

Non finisce mai: **0,1 è periodico in binario**, come 1/3 = 0,333… in decimale.

QUIZ A TRABOCCHETTO

# Quale numero si scrive in binario con un numero finito di cifre?

QUIZ

- **A** 0,1
- **B** 0,2
- **C** 0,75
- **D** 0,3

PRIMO MODO

# Virgola fissa: i posti sono decisi prima




8 bit: 4 per la parte intera, 4 dopo la virgola. `1010 1100` = 10,75

## Pro

Semplice e veloce: si calcola come con gli interi.

## Contro

Qui solo da 0 a 15,9375, a passi di 0,0625.

> Un'idea simile si usa con il denaro: molti programmi salvano gli importi come **numero intero di centesimi** (12,50 € → 1250), così non ci sono arrotondamenti.

SECONDO MODO

# La notazione scientifica fa "galleggiare" la virgola

Velocità della luce (m/s)

$2{,}998 \times 10^{8}$

Massa dell'elettrone (kg)

$9{,}109 \times 10^{-31}$

**Segno**

più o meno

**Mantissa**

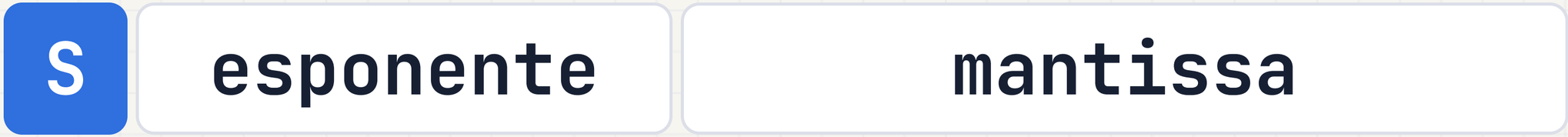
le cifre significative: 2,998

**Esponente**

dove va la virgola: 8 posti a destra

LO STANDARD

# IEEE 754: il float a 32 bit




Il valore

$$(-1)^{S} \times 1,\text{mantissa} \times 2^{\text{esponente} - 127}$$

| Formato | Segno | Esponente | Mantissa | Cifre decimali affidabili | In Java |
|---|---|---|---|---|---|
| Singola precisione | 1 | 8 | 23 | circa 7 | `float` |
| Doppia precisione | 1 | 11 | 52 | circa 15–16 | `double` |

A MANO

# −6,5 diventa C0D00000

1. Segno: negativo → **S = 1**
2. In binario: 6,5 = `110,1`
3. Normalizza: `1,101 × 2²`
4. Esponente: 2 + 127 = 129 = `1000 0001`
5. Mantissa: i bit dopo "1," → `101` e poi 20 zeri

Unisci e raggruppa a 4 bit

**1 10000001 101000…0 → 1100 0000 1101 0000 … → C0D00000**

AL COMPUTER · 15 minuti

# Dentro un float con il FloatConverter

1. Apri **h-schmidt.net/FloatConverter** (nessun account).
2. Scrivi **-6.5** (con il punto) e premi Invio: guarda segno, esponente e mantissa.
3. Prova **1**, **0.5** e **12.75**. Clicca sui bit per vedere come cambia il numero.
4. Scrivi **0.1** e leggi la riga "Value actually stored in float".
5. Annota i valori esadecimali in **float.txt**.

**Controllo:** -6.5 = `0xc0d00000`, 1 = `0x3f800000`, 12.75 = `0x414c0000`. Per 0.1 il float salva davvero **0.100000001490116…**

PRECISIONE

# Pochi bit, infiniti numeri: si arrotonda

- Tra 0 e 1 ci sono infiniti numeri reali, ma un float ha solo $2^{32}$ combinazioni
- Ogni numero viene salvato come il **più vicino** rappresentabile
- Gli errori piccoli si **sommano** nei calcoli lunghi

In doppia precisione

```
0,1 + 0,2 = 0,30000000000000004
```

I numeri rappresentabili sono fitti vicino allo zero e sempre più radi man mano che crescono.

AL COMPUTER · 10 minuti

# La console del browser fa i conti in double

Premi **F12**, scheda **Console**. Scrivi una riga alla volta e premi Invio:

```
0.1 + 0.2
0.1 + 0.2 === 0.3
Math.abs(0.1 + 0.2 - 0.3) < 1e-9
9007199254740993
1e308 * 10
```

Copia risultati e commenti in **float.txt**.

**Controllo:** 0.30000000000000004 · false · true · 9007199254740992 (l'ultima cifra cambia!) · Infinity

VALORI SPECIALI

# IEEE 754 ha anche numeri "strani"

**Infinity**

Esponente tutto a 1, mantissa a 0. Esiste anche -Infinity.

**NaN**, "Not a Number"

Il risultato non esiste. NaN non è uguale nemmeno a se stesso.

**Sì**: bit di segno a 1, tutto il resto a 0.

Ricordi i due zeri del modulo e segno?

PUZZLE

# Metti in ordine i passi per scrivere un float

1. Converti il valore assoluto in binario
2. Somma 127 all'esponente e scrivilo su 8 bit
3. Guarda il segno e scrivi il bit S
4. Copia i bit dopo "1," nei 23 bit di mantissa
5. Unisci i 32 bit e leggili in esadecimale
6. Normalizza nella forma 1,… × 2 e

STORIA VERA

# Dhahran, 25 febbraio 1991: un decimo di secondo

Il sistema antimissile Patriot contava il tempo in decimi di secondo, salvando 0,1 in binario con 24 bit: un piccolo errore a ogni tic.

Dopo circa 100 ore accesa, l'orologio era indietro di **0,34 secondi**. Il missile in arrivo non fu intercettato: morirono 28 soldati.

> **Il problema fondamentale della comunicazione è riprodurre in un punto, esattamente o approssimativamente, un messaggio scelto in un altro punto.**
>
> **Claude Shannon**, "A Mathematical Theory of Communication", 1948

AL COMPUTER · 15 minuti

# Dieci volte 0,1 in Java

```
public class Main {
  public static void main(String[] args) {
    double s = 0;
    for (int i = 0; i < 10; i++) {
      s = s + 0.1;           // 10 volte
    }
    System.out.println(s);
    System.out.println(s == 1.0);
    int centesimi = 0;
    for (int i = 0; i < 10; i++) {
      centesimi = centesimi + 10;
    }
    System.out.println(centesimi / 100.0);
  }
}
```

1. In **onlinegdb.com** scegli Java e copia il programma.
2. Prevedi le tre righe, poi premi **Run**.
3. Salva il codice in **DieciDecimi.txt**.

**Controllo:** 0.9999999999999999, poi false, poi 1.0: con i centesimi interi il conto torna.

METTITI ALLA PROVA

# In Java, come controlli se due double sono "uguali"?

QUIZ

A Con `a == b`, come per gli interi

B Con `Math.abs(a - b) < 1e-9`

C Convertendoli prima in int

D Usando float invece di double

OGGI

# La virgola mobile muove l'AI e i videogiochi





BIGLIETTO D'USCITA

# Completa prima di uscire

In binario, 0,5 + 0,25 si scrive 0,____.

Il numero 0,1 in binario ha infinite cifre: è _____.

In un float IEEE 754 l'esponente occupa _____ bit e si salva sommando 127.

Il risultato di 0.0 / 0 in Java è _____.

⏱ 3:00

RIEPILOGO

# Cosa portiamo a casa

- Dopo la virgola i pesi sono 0,5 · 0,25 · 0,125 …
- 0,1 è periodico in binario: va arrotondato
- Virgola fissa: semplice, ma poco intervallo
- Float: 1 + 8 + 23 bit, esponente + 127, "1," nascosto
- Double: 64 bit, circa 16 cifre affidabili
- Mai == tra reali: usa una tolleranza

Hai una domanda su una slide? Usa il pulsante **? Domande** in basso: la vedrà il docente e tutta la classe.